

IA NA PRÁTICA ACADÊMICA E PROFISSIONAL



01

QUINZENA 2

TOKENS, EMBEDDINGS E A
GEOMETRIA DO SIGNIFICADO

COMO O MODELO TRANSFORMA O QUE
VOCÊ ESCREVE ANTES DE RESPONDER

01

QUINZENA 2

IA NA PRÁTICA ACADÊMICA E PROFISSIONAL

TOKENS, EMBEDDINGS E A
GEOMETRIA DO SIGNIFICADO

COMO O MODELO TRANSFORMA O QUE
VOCÊ ESCREVE ANTES DE RESPONDER

INTRODUÇÃO



BASE CONCEITUAL DO MÓDULO 1

Este *e-book* articula o Módulo 1 do Workshop Fulbright sobre IA na Educação de Engenharia (Gerhardt, 2025), em especial a parte sobre representação de palavras em vetores, com o resultado clássico de Mikolov et al. (2013) sobre embeddings. Os exemplos de tokenização foram medidos em *tokenizadores* reais (GPT-4 e GPT-4o) para garantir números corretos.

Videoaula deste *e-book*: VA Q2-01 — Tokens: como a máquina lê o que você escreve.

Este módulo explica duas peças invisíveis que todo modelo de linguagem usa antes de gerar qualquer resposta: o **token** e o **embedding**. Você vai entender o que cada um é, por que eles existem e o que isso muda na forma como você usa a ferramenta. Os dois conceitos parecem técnicos, mas a ideia por trás deles é simples.

Quando você escreve uma pergunta para uma ferramenta de inteligência artificial (IA) e aperta **enter**, é natural imaginar que o modelo lê o seu texto do mesmo jeito que você: reconhece as palavras, entende as frases, acompanha os parágrafos. Não é isso que acontece. Antes de o modelo fazer qualquer coisa com a sua pergunta, o texto passa por uma transformação que muda tudo o que vem depois. Entender essa transformação é o primeiro passo para usar a ferramenta com consciência.

1. O MODELO NÃO LÊ TEXTO: ELE TRABALHA COM NÚMEROS

A primeira coisa a saber é que o modelo não trabalha com letras nem com palavras. Por dentro, ele só faz contas com números. Ele não tem nenhuma noção do que é a letra “a” ou a palavra “casa”. Para que o modelo possa processar a sua pergunta, **o texto precisa primeiro virar número.**

Para converter texto em número, existe uma espécie de “dicionário” preparado de antemão. Cada entrada desse dicionário é um pedaço de texto associado a um número. A palavra **casa** pode estar ligada ao número **1523**, o pedaço **cão**, ao número **8930**, e assim por diante. Esse dicionário fica pronto antes de você usar o modelo e não muda durante a conversa. Ele tem uma quantidade definida de entradas, em geral entre 30 mil e 100 mil.

Cada pedaço de texto desse dicionário se chama *token*. Um *token* costuma ser menor que uma palavra: pode ser uma **palavra curta inteira**, **uma sílaba**, **um sufixo**, **um sinal de pontuação** ou **um trecho no meio de uma palavra**. O modelo só consegue usar os pedaços que existem no dicionário, porque só esses têm um número. Se a sua palavra não estiver pronta lá dentro, ela é remontada com pedaços menores que existam. Essa quebra do texto em *tokens* se chama **tokenização**.

Você pode se perguntar por que esse dicionário não guarda simplesmente todas as palavras inteiras. O motivo é que não existe uma lista completa de palavras. Surgem nomes próprios, gírias, termos técnicos, erros de digitação e palavras de outros idiomas o tempo todo. Uma palavra que ficasse de fora viraria um vazio, algo que o modelo não conseguiria representar. O caminho oposto, guardar uma entrada para cada letra, resolveria esse problema, mas deixaria cada texto longo demais e lento de processar. O dicionário de pedaços é o meio-termo: **palavras muito comuns entram inteiras; palavras raras são montadas a partir de pedaços menores.** Assim, qualquer texto pode virar número, porque no pior caso o modelo recorre a pedaços bem pequenos, até letras isoladas.

A corrente completa funciona assim:

1. Você escreve um texto.
2. O texto é cortado em *tokens* que existem no dicionário.
3. Cada *token* vira um número.
4. Só então o modelo começa a trabalhar.

O QUE É UM TOKEN?

Token é um pedaço de texto.
Pode ser uma **palavra inteira**, **parte de uma palavra** ou **pontuação**.



! POR QUE ISSO IMPORTA

O modelo nunca vê letras soltas nem palavras inteiras do jeito que você vê. Ele vê tokens, que são pedaços de texto com um número associado. Quando um modelo parece não entender um termo técnico, muitas vezes é porque esse termo foi quebrado em pedaços que, separados, não carregam o sentido original.

2. A REGRA QUE DECIDE COMO O TEXTO É CORTADO

Falta responder qual regra define o corte. Por que “casa” pode virar um pedaço só e uma palavra rara é partida em vários? A regra mais usada tem um nome: **BPE**, sigla em inglês para *Byte Pair Encoding*, que se traduz como **codificação por pares**. O nome assusta, mas o procedimento é direto.

Antes de ser usado, o dicionário é construído uma única vez, a partir de uma quantidade enorme de texto. O processo começa com as letras separadas e repete um único movimento milhares de vezes: ele procura os dois pedaços que mais

aparecem juntos no texto e cola os dois num pedaço só. Em textos em inglês, as letras “t” e “h” aparecem grudadas com muita frequência, então viram o pedaço “th”. Em seguida, “th” e “e” aparecem juntos o tempo todo, então viram “the”. Esse movimento se repete até o dicionário atingir o tamanho desejado.

O efeito final é que as sequências mais comuns viram pedaços únicos, enquanto o que é raro continua quebrado em pedaços pequenos. Quando você escreve, o *tokenizador* apenas remonta o seu texto usando os maiores pedaços disponíveis no dicionário dele.

Isso explica por que nomes próprios se comportam de formas diferentes. Como o dicionário foi montado com muito texto em inglês e da internet, nomes e combinações de letras frequentes nesse material ganham um pedaço pronto. O nome “Ana” aparece com frequência suficiente para ser um único *token*. Já “Sebastião”, com a terminação “ão” e combinações mais raras em inglês, não existe pronto no dicionário, então é remontado em três pedaços: “Se, bast e ião”. O número de *tokens* de uma palavra reflete uma coisa só: quanto dela já existe pronto no dicionário. Quanto mais pronto, menos *tokens*.

A tabela abaixo mostra divisões reais feitas pelo tokenizador do GPT-4:

Palavra	Como o GPT-4 divide	Tokens
Ana	Ana	1
Maria	Maria	1
computador	comput / ador	2
inteligência	int / elig / ência	3
Sebastião	Se / bast / ião	3
alucinação	al / uc / ina / ção	4

3. MODELOS DIFERENTES CORTAM DE FORMAS DIFERENTES

Uma dúvida natural é se todos os modelos cortam o texto da mesma maneira. Não cortam. Cada família de modelos montou o seu dicionário com textos diferentes e com variações do método, então a mesma palavra pode ser dividida de formas distintas em cada um.

Um exemplo medido em *tokenizadores* reais deixa isso claro. O nome Ivanildo vira três pedaços no *tokenizador* do GPT-4 e apenas dois no do GPT-4o, um modelo

mais recente da mesma empresa. É a mesma palavra, processada por dois modelos, com contagens diferentes. Por isso, sempre que se fala em número de *tokens*, esse número vale para um modelo específico, não para a IA em geral.

4. POR QUE O PORTUGUÊS CUSTA MAIS QUE O INGLÊS

A forma como o dicionário foi montado tem uma consequência prática que afeta diretamente quem escreve em português. Como a maior parte do texto usado para construir o dicionário estava em inglês, o inglês é representado com poucos *tokens*. Já a mesma ideia escrita em português costuma ser cortada em mais pedaços.

Os números abaixo, medidos no *tokenizador* do GPT-4, mostram a diferença para uma frase equivalente nas duas línguas:

Língua	Frase	Tokens
Inglês	Artificial intelligence is transforming education	6
Português	A inteligência artificial está transformando a educação	10

Mais *tokens* significam mais processamento e mais custo. Significam também menos espaço na janela de contexto, que é **o limite de *tokens* que o modelo consegue considerar de uma vez**. Quem usa apenas a interface não percebe nada disso, mas a diferença é real. Ela não é um defeito de programação de um modelo específico: é uma característica estrutural, resultado de quais textos foram usados para montar o dicionário.

5. EMBEDDINGS: O ENDEREÇO DE CADA TOKEN NUM MAPA DE SIGNIFICADOS

Dividir o texto em *tokens* e transformá-los em números resolve só a entrada. Falta o modelo saber o que cada *token* significa em relação aos outros. Um número de dicionário, sozinho, não diz nada sobre sentido: o número 1523 da palavra casa não tem relação nenhuma com o número 8930 do pedaço ção. É aqui que entra o segundo conceito do módulo, o **Embeddings**.

Cada *token* recebe um “endereço”. Não um endereço de rua nem um CEP, mas um conjunto de números que marca uma posição dentro de um mapa. Esse mapa tem

muitas dimensões, em geral centenas ou milhares. Um modelo recente como o **Qwen**¹ usa cerca de 17 mil dimensões. Não é preciso imaginar esse espaço visualmente; basta entender o que a posição representa. Esse endereço numérico de cada token é o que se chama **embedding**.

COMO IMAGINAR UM MAPA COM MUITAS DIMENSÕES

Uma dimensão é só uma informação que você mede. Marcar a temperatura de uma cidade usa uma dimensão, um único número numa linha. Marcar um ponto num mapa de papel usa duas, latitude e longitude. Acrescentar a altitude usa três. **Acima de três ninguém consegue visualizar**, nem precisa: basta pensar num ponto como uma lista de números, em que cada número descreve uma característica.

Imagine descrever frutas por três características: o quanto são doces, o tamanho e o quanto são vermelhas. Cada fruta vira uma lista de três números, e duas frutas ficam próximas quando suas listas são parecidas. O modelo faz o mesmo com as palavras, só que usa centenas ou milhares de características que ele mesmo descobriu. O *embedding* é essa lista de números, e a proximidade no mapa quer dizer semelhança entre as listas.

A regra desse mapa é simples de enunciar: *tokens* que aparecem em contextos parecidos ficam em posições próximas. As palavras rei e rainha ficam perto uma da outra, porque aparecem em frases parecidas. Pizza e macarrão também ficam próximas. Palavras sem relação ficam distantes.

O ponto que muda tudo é como o modelo chegou a essas posições. Ninguém ensinou ao modelo que rei e rainha se relacionam. Ele aprendeu isso sozinho, observando em que contextos essas palavras aparecem juntas ao longo de bilhões de frases. A proximidade no mapa é resultado de padrão estatístico, não de uma definição que alguém escreveu. O modelo não consultou um dicionário de significados; ele percebeu regularidades no uso das palavras e as transformou em posições.



ANALOGIA PARA FIXAR

Pense no *token* como um pedaço de texto com um número e no *embedding* como o endereço desse pedaço num mapa de significados. Dois pedaços que costumam aparecer nos mesmos contextos ganham endereços vizinhos. O modelo desenhou esse mapa observando o uso real das palavras, sem nenhuma regra de gramática escrita à mão.

1. O **Qwen** é um modelo de linguagem de código aberto da **Alibaba**, uma empresa de tecnologia chinesa. Por ser aberto, qualquer pessoa pode examinar como ele foi construído por dentro, incluindo quantas dimensões usa para representar cada token. É por isso que ele costuma aparecer em explicações técnicas

6. A ARITMÉTICA DO SIGNIFICADO

Uma demonstração clássica mostra até onde vai essa organização por posição. Mikolov et al. (2013), perceberam que dá para fazer contas com os endereços das palavras e obter resultados com sentido. Partindo do endereço de rei, subtraindo o de homem e somando o de mulher, o ponto de chegada no mapa é o endereço de rainha. Essa conta funciona com outras palavras: o endereço de Paris menos o de França mais o de Itália chega perto do endereço de Roma.

Conta feita com os endereços	Aonde se chega no mapa
rei – homem + mulher	rainha
Paris – França + Itália	Roma

Esse resultado é revelador porque mostra que o modelo não guardou definições prontas. Ele guardou relações entre palavras na forma de geometria, isto é, na forma de posições e distâncias dentro do mapa. A relação entre um país e a sua capital ou entre masculino e feminino fica registrada na posição relativa entre os pontos.

7. AS TRÊS CAMADAS E O QUE VEM A SEGUIR

Vale juntar as peças. Quando você envia uma pergunta, o modelo passa a lidar com três representações do mesmo texto ao mesmo tempo. A primeira é a frase como texto, do jeito que você escreveu. A segunda são os *tokens*, os pedaços em que a frase foi cortada. A terceira é a posição de cada *token* no mapa de significados, o *embedding*. **É a partir dessas três camadas que o modelo começa o que, por fora, parece leitura.**

O que ainda falta é o passo seguinte: como o modelo decide qual *token* vem depois, levando em conta todos os outros da frase. Esse mecanismo se chama **atenção** e é o tema do próximo módulo. Por enquanto, basta ter claro que tudo começa com tokens e *embeddings*.

8. O QUE ISSO MUDA NO SEU USO

Entender ***tokens*** e ***embeddings*** ajuda a interpretar comportamentos do modelo que, sem esse conhecimento, parecem aleatórios. **Quando o modelo erra um nome próprio, confunde uma abreviação ou trata um termo técnico de**

forma estranha, em geral há uma explicação na forma como o texto foi cortado e representado. Um termo raro foi quebrado em pedaços que, separados, perderam o sentido; um nome pouco frequente no material de treino não tinha um endereço bem definido no mapa.

Fenômeno observado	O que está acontecendo por dentro
O modelo erra ou deforma um nome próprio	O nome é raro no material de treino e foi quebrado em pedaços sem sentido próprio, sem um endereço bem definido no mapa.
O modelo usa mal um termo técnico em português	O termo foi cortado em fragmentos que, separados, perdem o significado original. Termos incomuns são os mais afetados.
A mesma frase custa mais em português que em inglês	O dicionário foi montado com predominância de inglês, então o português é cortado em mais tokens.

Reconhecer esses limites tem um nome dentro desta disciplina: desnaturalizar a tecnologia. Significa enxergar o que a interface esconde e entender que a ferramenta, que parece mágica, tem estrutura, lógica e limites que fazem sentido quando você sabe o que existe por dentro dela. Esse é o ponto de partida para tudo o que vem nos próximos módulos.



PARA REFLEXÃO

Na atividade deste módulo, você vai colocar o seu próprio nome em uma ferramenta de *tokenização* e observar em quantos pedaços ele é dividido. Depois vai comparar com palavras em inglês e anotar o que mudou. A pergunta que orienta a atividade não é quantos *tokens* apareceram, e sim o que essa divisão revela sobre como o modelo enxerga aquilo que você escreve.

REFERÊNCIAS

As obras abaixo são verificáveis e estão disponíveis nas plataformas indicadas.

KATZ, B. *et al.* Integrating artificial intelligence into engineering education. *In: WORKSHOP FULLBRIGHT.*, 2025, São Paulo. **Anais** [...], São Paulo: Centro de Inovação da USP, 2025. Módulo 1: Gerhardt, M. Introduction to Generative AI.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. **Deep learning**. Cambridge: MIT Press, 2016. Disponível em: <https://www.deeplearningbook.org>. Acesso em: 2 jul. 2026.

MIKOLOV, T. *et al.* Efficient Estimation of Word Representations in Vector Space. **arXiv**, 2013. Disponível em: <https://arxiv.org/abs/1301.3781>. Acesso em: 2 jul. 2026.

CRÉDITOS

Autor Principal: **Prof. Dr. José Avelino Placca**
Coordenação: **Wesley de Souza Lima**
Design Instrucional: **Claudia Mori**
Revisão Textual: **Juliana Nascimento Costa e Valteir Vaz**
Edição de Arte: **Henrique Inhauser Caldas**

Título da obra:
O que é (e o que não é) IA

Local de publicação:
São Paulo, Brasil

Instituição: **Univesp**
Ano de publicação: **2026**
Número de páginas: **12 p.**
Palavras-chave: **1. Inteligência Artificial. 2. Aprendizado de Máquina. 3. IA Generativa**



ATRIBUIÇÃO – NÃO COMERCIAL (BY - NC - ND)

PROIBIDA A EDIÇÃO. PROIBIDO USO COMERCIAL. PERMITE SOMENTE REDISTRIBUIÇÃO. PERMITE O DOWNLOAD E COMPARTILHAMENTO, CONTANTO QUE O AUTOR SEJA MENCIONADO E A OBRA INALTERADA.